

Ubuntu・OS基礎 練習課題と解答例

P1～P6 課題と操作手順

Ubuntu Server 24.04 LTS / 公開Lab v1.0.0
JDU Cyber Security / 2026-09-25

Ubuntu and OS Foundations — 練習課題と解答例

対象: 練習課題P1～P6

環境: Ubuntu Server 24.04 LTS

進め方: まず各Pの「課題」を読む。自分で試した後、「解答例」の手順で操作と考え方を確認する。同じ技能を使うMission Mは自力で解く。

最初にUbuntuへ接続する

AWS Academy Learner Labのセットアップを完了してから、AWSコンソールでCloudShellを開く。CloudShellで次を実行して、自分のUbuntu演習サーバーへ接続する。

```
ssh jdu-ubuntu
```

P1は、接続先のUbuntuで進める。CloudShell側の操作が必要なP6では、手順中に実行場所を示す。

0. 共通ルール

P1～P6は練習用である。M1～M6とは別のdirectory、user、group、service、portを使う。M7は統合課題であるため、対応する練習課題を設けない。

各Pには、最初に問題文、その後に解答例を置く。問題文だけを読んで解いてもよい。解答例の手順を見ながら進めてもよい。PとMの採点対象は別であり、PのPASSはMのPASSにならない。

Ubuntuで次を実行する。表示がssm-userであることを確認する。

```
id -un
```

別のuser名が表示された場合は、exitを1回実行する。もう一度確認する。

```
exit  
id -un
```

各手順では、最初に作業directoryへ移動する。移動後にpwdで現在地を確認する。

Ubuntu側の判定は、ssm-userで次の形で実行する。P6のCloudShell側の判定は、その環境で実行する。

```
jdu-check P1
```

判定は課題の完成状態を調べる。観察しただけの操作や途中の操作は、判定項目に含まれない場合がある。判定結果は設定済みの教員用進捗画面へ自動送信される。送信できなくても、UbuntuまたはCloudShell上のPASS/FAILは確認できる。

最初からやり直す場合だけresetする。

```
jdu-reset P1
```

P1 Shell、path、file、text

学ぶこと

- Home directoryと作業directoryを区別する。
- Directoryを作る。
- Fileをコピーする。
- Logから必要な行を保存する。

課題

~/jdu-lab/p1/inboxに設定ファイルとログがある。~/jdu-lab/p1/practice01は未完成で、不要な.tmpファイルが残っている。次の状態に仕上げる。

1. practice01の下にconfig、logs、notesの3つのディレクトリを作る。
2. inbox/config/training.confをpractice01/config/training.confへ、inbox/logs/practice.logをpractice01/logs/practice.logへコピーする。元ファイルとコピーの内容を変えない。
3. practice01以下の.tmpファイルをすべて削除する。それ以外のファイルは消さない。
4. 完成したpractice01以下のディレクトリとファイルの所有者を、Ubuntuの管理ユーザーssm-userにする。
5. コピーしたログからWARNを含む行だけを、行番号を付けずにpractice01/notes/warnings.txtへ保存する。
6. 同じログの最後の4行を、元の順序でpractice01/notes/recent.txtへ保存する。

完成状態をjdu-check P1で判定する。6項目すべてがPASSなら終了する。

解答例（操作手順）

手順1: Home directoryへ移動する

```
cd ~  
pwd
```

表示が/home/ssm-userであることを確認する。

手順2: P1 directoryへ移動する

```
cd ~/jdu-lab/p1  
pwd  
tree
```

inboxと、未完成のpractice01を確認する。

手順3: 必要なdirectoryを作る

```
cd ~/jdu-lab/p1  
mkdir -p practice01/config  
mkdir -p practice01/logs  
mkdir -p practice01/notes  
tree practice01
```

手順4: Config fileをコピーする

コピー先のdirectoryへ移動する。

```
cd ~/jdu-lab/p1/practice01/config
pwd
cp .././inbox/config/training.conf .
ls -l
```

.は現在のdirectoryを表す。

手順5: Log fileをコピーする

```
cd ~/jdu-lab/p1/practice01/logs
pwd
cp .././inbox/logs/practice.log .
ls -l
```

手順6: 一時fileを確認して削除する

practice01へ移動し、削除前の構成を確認する。

```
cd ~/jdu-lab/p1/practice01
pwd
tree
```

treeの表示から、stagingの中にあるtraining.conf.tmpとpractice.log.tmpを探す。次に、同じ2つのfileをfindで検索する。

```
find . -type f -name '*.tmp' -print
```

.は現在のdirectoryから探す指定、-type fは通常のfileだけを対象にする条件、-name '*.tmp'は名前が.tmpで終わる条件、-printは一致したパスを表示する指示である。*.tmp'を引用符で囲む理由を含め、詳しい読み方は巻末の参照資料で説明する。

./staging/training.conf.tmpと./staging/practice.log.tmpが表示されることを確認する。表示順は問わない。検索結果とtreeの構成を照合したら、この2つの不要なfileをrmで削除する。

```
rm staging/training.conf.tmp staging/practice.log.tmp
tree
```

削除後のtreeで、2つの.tmpファイルが表示されないことを確認する。コピーしたconfig/training.confとlogs/practice.logが残っていることも確認する。

手順7: WARN行を確認して保存する

出力先のnotesへ移動する。まず、grepの結果を画面で確認する。

```
cd ~/jdu-lab/p1/practice01/notes
pwd
grep 'WARN' ../logs/practice.log
```

WARNを含む行だけが表示されることを確認する。次に、同じcommandに>を追加して、結果をwarnings.txtへ保存する。>は出力先を画面からfileへ変更し、同名のfileがあれば内容を上書きする。

```
grep 'WARN' ../logs/practice.log > warnings.txt
cat warnings.txt
```

catで、先ほど画面に表示された行が保存されていることを確認する。

手順8: 最後の4行を確認して保存する

まず、tailの結果を画面で確認する。

```
cd ~/jdu-lab/p1/practice01/notes
tail -n 4 ../logs/practice.log
```

ログの最後の4行が、元の順序で表示されることを確認する。次に、>を追加して結果をrecent.txtへ保存する。

```
tail -n 4 ../logs/practice.log > recent.txt
cat recent.txt
```

catで、先ほど画面に表示された4行が保存されていることを確認する。

手順9: 全体を確認する

```
cd ~/jdu-lab/p1
tree practice01
jdu-check P1
```

全6件がPASSになったら、M1へ進む。

P2 User、group、permission、setgid

学ぶこと

- Primary groupとsupplementary groupを確認する。
- 共有directoryへgroup permissionを設定する。
- setgidで新しいfileのgroupを継承する。

課題

共有ディレクトリ/srv/jdu-practice-shareを、書き込み担当者と閲覧担当者と使い分ける。初期状態では、jdupracticewriterが補助グループpracticeopsに入っており、書き込み担当のjdupracticewriterは入っていない。ディレクトリとGUIDE.txtの所有者・グループ・権限も未完成である。

1. jdupracticewriterを補助グループpracticeopsに追加し、jdupracticewriterをそのグループから削除する。
2. /srv/jdu-practice-shareの所有者とグループをroot:practiceops、権限を2775にする。新しいファイルが共有グループを引き継ぐようにする。
3. GUIDE.txtの所有者とグループをroot:practiceops、権限を664にする。
4. ssm-userからjdupracticewriterへ切り替えて、共有ディレクトリにwriter-created.txtを作る。新しいファイルのグループがpracticeopsであることを確認する。
5. ssm-userに戻り、jdupracticewriterへ切り替える。GUIDE.txtは読めるが、共有ディレクトリに新しいファイルは作れないことを確認する。

異なるユーザーでの操作を終えたらssm-userへ戻り、jdu-check P2を実行する。グループ所属、ディレクトリ、ファイル、作成時のグループ継承、閲覧担当者のアクセスを計6項目で判定する。

解答例（操作手順）

手順1: 管理userを確認する

```
cd ~  
id -un  
id
```

手順2: 初期groupを確認する

```
getent group practiceops  
id jdupracticewriter  
id jdupracticeviewer
```

初期状態ではjdupracticeviewerがpracticeopsに入り、jdupracticewriterが入っていない。

手順3: Group membershipを修正する

```
cd ~  
sudo usermod -aG practiceops jdupracticewriter  
sudo gpasswd -d jdupracticeviewer practiceops  
getent group practiceops  
id jdupracticewriter  
id jdupracticeviewer
```

手順4: 共有directoryを設定する

```
cd /srv  
pwd  
sudo chown root:practiceops jdu-practice-share  
sudo chmod 2775 jdu-practice-share  
stat -c '%U:%G %a %n' jdu-practice-share
```

手順5: GUIDE.txtを設定する

対象directoryへ移動する。

```
cd /srv/jdu-practice-share  
pwd  
sudo chown root:practiceops GUIDE.txt  
sudo chmod 664 GUIDE.txt  
stat -c '%U:%G %a %n' GUIDE.txt
```

手順6: Writerへ切り替えてfileを作る

```
cd ~  
sudo su - jdupracticewriter  
id -un  
id  
cd /srv/jdu-practice-share  
pwd  
printf '%s\n' 'guided writer file' > writer-created.txt  
ls -l writer-created.txt  
exit  
id -un
```

最後にssm-userと表示されることを確認する。

手順7: Viewerのreadとwriteを確認する

```
cd ~
sudo su - jdupracticeviewer
id -un
id
cd /srv/jdu-practice-share
pwd
cat GUIDE.txt
touch viewer-created.txt
exit
id -un
```

catは成功する。touchはPermission deniedになる。これは想定した失敗である。

手順8: 採点する

```
cd ~
id -un
jdu-check P2
```

全6件がPASSになったら、M2へ進む。

P3 Processとpackage

第8章を読んだら手順1〜4でプロセスを確認・停止する。続いて第9章を読み、手順5〜6でパッケージを導入する。両方を終えてからjdu-check P3を実行する。

学ぶこと

- systemd serviceとprocessのPIDを対応させる。
- PIDを指定してTERM signalを送る。
- aptでpackageをinstallする。

課題

初期状態でjdu-p3-process1.service、jdu-p3-process2.service、jdu-p3-process3.serviceが動いている。figletは未インストールである。次のAとBの両方を完了する。

課題A - process2だけを停止する。jdu-p3-process2.serviceのMain PIDを調べ、そのPIDのプロセスをpsで確かめる。PIDを指定してTERMシグナルを送る。process2だけが停止し、process1とprocess3は動いたままにする。名前の広い条件で複数プロセスを止めたり、KILLシグナルを使ったりしない。

課題B - パッケージを導入する。figletのパッケージ情報を調べ、aptでインストールする。コマンドを1回実行し、インストール済みのバージョンと/usr/bin/figletを提供するパッケージを確認する。

jdu-check P3でAとBを各1項目、計2項目として判定する。調査時のコマンド履歴やメモは提出しない。

解答例（操作手順）

手順1: 3 serviceを確認する

```
cd ~
systemctl list-units --type=service 'jdu-p3-*'
systemctl status jdu-p3-process1.service --no-pager
systemctl status jdu-p3-process2.service --no-pager
systemctl status jdu-p3-process3.service --no-pager
```

手順2: Process2のMain PIDを表示する

```
systemctl show --property MainPID --value jdu-p3-process2.service
```

手順3: PIDをshell変数へ保存する

```
P3_PID=$(systemctl show --property MainPID --value jdu-p3-process2.service)
printf '%s\n' "$P3_PID"
ps -fp "$P3_PID"
```

手順4: Process2だけへTERMを送る

```
sudo kill -TERM "$P3_PID"
systemctl is-active jdu-p3-process1.service
systemctl is-active jdu-p3-process2.service
systemctl is-active jdu-p3-process3.service
```

順に **active**、**inactive**、**active** になる。

手順5: figletを調査してinstallする

```
cd ~
apt show figlet
sudo apt update
sudo apt install -y figlet
command -v figlet
figlet JDU
```

手順6: Packageとcommandの対応を確認する

```
dpkg-query -W -f='${Status} ${Version}\n' figlet
dpkg -S /usr/bin/figlet
jdu-check P3
```

全2件がPASSになったら、M3へ進む。

P4 systemd service

学ぶこと

- Unit fileを読む。
- Activeとenabledを別々に設定する。
- Main PIDと実processを対応させる。

課題

jdu-practice-status.serviceのユニットファイルは完成しているが、サービスは停止中で、自動起動も無効である。ユニットファイルを変更せずに次を行う。

1. ユニットファイルのUser、WorkingDirectory、ExecStartを調べる。
2. サービスを起動し、現在の状態がactiveであることを確認する。
3. 起動時の自動実行を有効にし、enabledであることを確認する。activeとenabledは別の状態として調べる。
4. Main PIDのプロセスについて、実行ユーザー、コマンドライン、作業ディレクトリがユニットファイルと一致することを確かめる。

jdu-check P4は、稼働状態、自動起動、変更されていないユニットと実プロセスの対応を計3項目で判定する。観察結果を別ファイルへ提出する必要はない。

解答例（操作手順）

手順1: Unit fileを読む

```
cd ~
systemctl cat jdu-practice-status.service
```

User、WorkingDirectory、ExecStartを確認する。

手順2: 初期状態を確認する

```
systemctl is-active jdu-practice-status.service
systemctl is-enabled jdu-practice-status.service
```

手順3: Serviceをstartする

```
sudo systemctl start jdu-practice-status.service
systemctl is-active jdu-practice-status.service
```

手順4: Boot時にstartするよう設定する

```
sudo systemctl enable jdu-practice-status.service
systemctl is-enabled jdu-practice-status.service
```

手順5: Main PIDとprocessを確認する

```
P4_PID=$(systemctl show --property MainPID --value jdu-practice-status.service)
printf '%s\n' "$P4_PID"
ps -fp "$P4_PID"
sudo cat "/proc/$P4_PID/cmdline" | tr '\0' ' '
printf '\n'
sudo readlink -f "/proc/$P4_PID/cwd"
```

手順6: 採点する

```
cd ~
jdu-check P4
```

全3件がPASSになったら、M4へ進む。

P5 Port、socket、HTTP、journal

学ぶこと

- Service、process、listening socketを対応させる。
- IP addressとportを読む。
- HTTP requestがjournalへ記録されることを確認する。

課題

jdu-practice-web.serviceは停止中である。ユニットファイルを変更せず、次の一連のつながりを完成させる。

1. ユニットファイルから、実行ユーザー、公開するファイルのパス、待受IPアドレスとポート番号を調べる。サービスを起動する。
2. 127.0.0.1:8181で待ち受けているソケットを見つける。そのソケットのPIDがサービスのMain PIDと同じであることを確かめる。全アドレスでの待受には変更しない。
3. http://127.0.0.1:8181/へリクエストを送り、HTTP 200と本文を確認する。サービスの実行ユーザーが/srv/jdu-practice-web/index.txtを読めることも確かめる。
4. http://127.0.0.1:8181/p5-checkへ自分でリクエストを送り、現在のサービスのシステムジャーナルにREQUEST path=/p5-checkが記録される状態にする。
5. ポート18181には待受がないことを調べ、接続できる8181との違いを確認する。

jdu-check P5は、ソケット、HTTPとファイルアクセス、指定リクエストのログ、開いているポートと閉じているポートの比較を計4項目で判定する。

解答例（操作手順）

手順1: Unit fileを読む

```
cd ~
systemctl cat jdu-practice-web.service
```

127.0.0.1、8181、jdupracticeweb、content fileのpathを確認する。

手順2: Serviceをstartする

```
sudo systemctl start jdu-practice-web.service
systemctl is-active jdu-practice-web.service
```

手順3: Listening socketを確認する

```
sudo ss -ltnp | grep ':8181'
```

127.0.0.1:8181を確認する。同じ出力で0.0.0.0:8181や[*]:8181の待受がないことも確認する。前者はこのホスト内からの接続だけを受け付ける。後者は全アドレスでの待受を表す。

手順4: Socket PIDとMain PIDを比較する

```
systemctl show --property MainPID --value jdu-practice-web.service
sudo ss -ltnp | grep ':8181'
```

2つのPIDが同じであることを確認する。

手順5: HTTP responseを確認する

```
curl -i http://127.0.0.1:8181/
```

手順6: Service userのread権限を確認する

Content directoryへ移動する。

```
cd /srv/jdu-practice-web
pwd
stat -c '%U:%G %a %n' index.txt
namei -l /srv/jdu-practice-web/index.txt
id jdupracticeweb
sudo -u jdupracticeweb -- test -r index.txt
echo $?
```

最後の値0はread可能を表す。

手順7: 学生用requestを送る

```
curl -i http://127.0.0.1:8181/p5-check
sudo journalctl -u jdu-practice-web.service --no-pager -n 20
```

REQUEST path=/p5-checkを確認する。

手順8: Closed portと比較する

```
curl --max-time 2 http://127.0.0.1:18181/
sudo ss -lnt | grep ':18181'
```

どちらも成功しない。TCP 18181は待ち受けられていない。

手順9: 採点する

```
cd ~
jdu-check P5
```

全4件がPASSになったら、M5へ進む。

P6 SSH、remote command、scp

学ぶこと

- CloudShellとUbuntuを区別する。
- scpでuploadとdownloadを行う。
- SSH remote commandを実行する。

課題

CloudShellを接続元、Ubuntuを接続先として作業する。SSHの接続設定とjdu-ubuntuという接続名は準備済みである。P6の転送ファイルはまだない。

リモートで作る結果ファイルは、次の3行の形式にする。右辺は実際のコマンド出力を使い、推測して手入力しない。

```
REMOTE_USER=実際のユーザー名  
REMOTE_HOST=実際のホスト名  
REMOTE_PATH=実際のホームディレクトリ
```

1. CloudShellの~/jdu-lab/p6/practice-source.txtへ、JDU SSH guided transferの1行だけを書き込む。
2. scpでそのファイルをUbuntuの~/jdu-lab/p6/practice-upload.txtへ送る。転送前後のSHA-256が同じことを確認する。
3. CloudShellからSSHでUbuntu上のコマンドを実行し、Ubuntuの~/jdu-lab/p6/practice-remote-result.txtへ、実際のリモートユーザー・ホスト名・ホームディレクトリを上形式で保存する。
4. Ubuntuへ接続してjdu-check P6を実行する。Ubuntu側の2項目がPASSになることを確認する。
5. 結果ファイルをscpでCloudShellの~/jdu-lab/p6/practice-downloaded-result.txtへ戻す。両側のSHA-256が同じことを確認する。
6. CloudShellでjdu-check P6を実行する。CloudShell側の4項目がPASSになることを確認する。

P6はUbuntu側2/2とCloudShell側4/4の両方が揃って完了する。教員用進捗画面でも両側を区別して表示する。

解答例（操作手順）

手順1: UbuntuからCloudShellへ戻る

Ubuntuのpromptで実行する。

```
exit
```

CloudShellで現在地を確認する。

```
cd ~  
pwd  
id -un  
hostname
```

手順2: CloudShell側の作業directoryを作る

```
mkdir -p ~/jdu-lab/p6  
cd ~/jdu-lab/p6  
pwd
```

手順3: Upload元fileを作る

```
printf '%s\n' 'JDU SSH guided transfer' > practice-source.txt  
cat practice-source.txt  
sha256sum practice-source.txt
```

手順4: Ubuntu側のdirectoryを作る

CloudShellで実行する。

```
ssh jdu-ubuntu 'mkdir -p ~/jdu-lab/p6'
```

手順5: Ubuntuへuploadする

CloudShellのP6 directoryにいる状態で実行する。

```
cd ~/jdu-lab/p6
scp practice-source.txt jdu-ubuntu:~/jdu-lab/p6/practice-upload.txt
ssh jdu-ubuntu 'sha256sum ~/jdu-lab/p6/practice-upload.txt'
sha256sum practice-source.txt
```

2つのSHA-256が同じであることを確認する。

手順6: SSH remote commandで結果fileを作る

CloudShellで、次の1 commandをそのまま実行する。

```
ssh jdu-ubuntu 'cd ~/jdu-lab/p6 && printf "REMOTE_USER=%s\nREMOTE_HOST=%s\nREMOTE_PATH=%s\n" "$(id -un)"
"${hostname}" "$HOME" > practice-remote-result.txt'
```

内容をremoteで確認する。

```
ssh jdu-ubuntu 'cat ~/jdu-lab/p6/practice-remote-result.txt'
```

手順7: Ubuntu側を採点する

Ubuntuへ接続する。

```
ssh jdu-ubuntu
cd ~/jdu-lab/p6
pwd
ls -l
jdu-check P6
exit
```

Ubuntu側2件がPASSになる。

手順8: Result fileをdownloadする

CloudShellで実行する。

```
cd ~/jdu-lab/p6
scp jdu-ubuntu:~/jdu-lab/p6/practice-remote-result.txt practice-downloaded-result.txt
cat practice-downloaded-result.txt
sha256sum practice-downloaded-result.txt
ssh jdu-ubuntu 'sha256sum ~/jdu-lab/p6/practice-remote-result.txt'
```

2つのSHA-256が同じであることを確認する。

手順9: CloudShell側を採点する

```
cd ~/jdu-lab/p6
jdu-check P6
```

CloudShell側4件がPASSになったら、M6へ進む。

学習順序

教科書の章は授業回数と一対一ではない。理解した範囲から演習へ進み、必要なら章へ戻る。

- ・第1～3章と第4章の編集の基本を読む。その後、M0で実機のOSを観察する。
- ・第4・5章を読んでP1、M1へ進む。

- 第6・7章を読んでP2、M2へ進む。
- 第8章を読んでP3手順1～4を行う。第9章を読んでP3手順5～6を行い、最後にM3へ進む。
- 第10章を読んでP4、M4へ進む。第11章を読んでP5、M5へ進む。
- 第12章を読んでP6、M6へ進む。既習の操作を組み合わせでM7へ進む。

演習全体では次の順序を推奨する。

```
P1 → M1 → P2 → M2 → P3 → M3 → P4 → M4
→ P5 → M5 → P6 → M6 → M7 (統合課題)
```

Pは問題文だけで試してから解答例を見ることも、解答例を見ながら進めることもできる。M1～M6ではPの手順を見ずに、必要なcommandを自分で選ぶ。M7では、M2、M4、M5で学んだ内容を統合する。